

Hdoc Day-20

Name - Aaryan Pratap Singh

Sap id- 590032842

Batch - 15

Application of Iterative and Decision-Making Constructs to Shapes and Physics Using Modular Programming in C

1. Project Modules

File	Responsibility
main.c	Displays menu, validates common inputs, calls functions, displays results
geometry.c	Calculates surface area, volume, base area and cone slant height
physics.c	Calculates mass, weight, potential energy, pressure, buoyant force and buoyancy behaviour
math_utils.c	Calculates square root using Newton-Raphson iteration
solid.h	Header guard, constants, solid codes, buoyancy codes and function declarations

2. Algorithms

Algorithm 1 — Main Program

1. Start.
2. Display the main menu:
 1. Surface Area
 2. Volume
 3. Base Area
 4. Mass

5. Weight
 6. Gravitational Potential Energy
 7. Pressure on Base
 8. Buoyant Force
 9. Buoyancy Behaviour
 10. Exit
3. Read the user's choice.
 4. If the choice is outside 1–10, display an error and repeat the menu.
 5. Use **switch-case** to perform the selected operation.
 6. For geometry operations, obtain and validate the solid type and dimensions.
 7. For physical operations, obtain and validate the required density/elevation and call the appropriate physics function.
 8. Display the result with the appropriate SI unit.
 9. Repeat using a **do-while** loop until option 10 is selected.
 10. Stop.

Algorithm 2 — Sphere Surface Area

Function: `sphere_area(radius)`

1. Receive radius.
2. Calculate

$$\text{area} = 4 \times \text{PI} \times \text{radius} \times \text{radius}$$
3. Return area.

Sphere Volume

1. Receive radius.
2. Calculate

$$\text{volume} = (4/3) \times \text{PI} \times \text{radius}^3$$
3. Return volume.

Algorithm 3 — Cylinder

Surface Area

1. Receive radius and height.
2. Calculate

$$\text{area} = 2 \times \text{PI} \times \text{radius} \times (\text{radius} + \text{height})$$
3. Return area.

Volume

1. Receive radius and height.
2. Calculate

$$\text{volume} = \text{PI} \times \text{radius}^2 \times \text{height}$$
3. Return volume.

Algorithm 4 — Cone

Surface Area

1. Receive radius and height.
2. Calculate $\text{radius}^2 + \text{height}^2$.
3. Call `calculate_square_root()` to obtain slant height.
4. Calculate
 $\text{area} = \text{PI} \times \text{radius} \times (\text{radius} + \text{slant_height})$
5. Return area.

Volume

1. Receive radius and height.
2. Calculate
 $\text{volume} = (1/3) \times \text{PI} \times \text{radius}^2 \times \text{height}$
3. Return volume.

Algorithm 5 — Cuboid

Surface Area

1. Receive length, breadth and height.
2. Calculate
 $\text{area} = 2 \times (\text{length} \times \text{breadth} + \text{breadth} \times \text{height} + \text{length} \times \text{height})$
3. Return area.

Volume

1. Receive length, breadth and height.
2. Calculate
 $\text{volume} = \text{length} \times \text{breadth} \times \text{height}$
3. Return volume.

Algorithm 6 — Base Area

Cylinder/Cone

1. Receive radius.
2. Calculate
 $\text{base_area} = \text{PI} \times \text{radius}^2$
3. Return base area.

Cuboid

1. Receive length and breadth.
2. Calculate
 $\text{base_area} = \text{length} \times \text{breadth}$

3. Return base area.

For a sphere, display that the ideal geometrical base area is not defined and return to the main menu.

Algorithm 7 — Mass

1. Receive density.
2. Obtain the solid type.
3. Validate solid type.
4. Obtain and validate its required dimensions.
5. Calculate volume by calling the corresponding geometry function.
6. Calculate
$$\text{mass} = \text{density} \times \text{volume}$$
7. Return mass.

Algorithm 8 — Weight

1. Obtain and validate density.
2. Call `calculate_mass()`.
3. Calculate
$$\text{weight} = \text{mass} \times \text{GRAVITY}$$
4. Return weight.

Algorithm 9 — Gravitational Potential Energy

1. Receive validated elevation.
2. Obtain and validate density.
3. Call `calculate_mass()`.
4. Calculate
$$\text{PE} = \text{mass} \times \text{GRAVITY} \times \text{elevation}$$
5. Return potential energy.

Algorithm 10 — Pressure on Base

1. Receive density.
2. Obtain the solid type.
3. If sphere is selected, display that pressure on the base cannot be calculated and request another supported solid.
4. For cylinder/cone:
 - Obtain radius and height.
 - Calculate volume.
 - Calculate circular base area.
5. For cuboid:
 - Obtain length, breadth and height.
 - Calculate volume.
 - Calculate cuboid base area.
6. Calculate
$$\text{pressure} = \text{density} \times \text{volume} \times \text{GRAVITY} / \text{contact_area}$$

7. Return pressure.

Algorithm 11 — Buoyant Force

1. Receive fluid density.
2. Obtain the solid type.
3. Obtain and validate dimensions.
4. Calculate the solid's volume.
5. Calculate
 $\text{buoyant_force} = \text{fluid_density} \times \text{volume} \times \text{GRAVITY}$
6. Return buoyant force.

Algorithm 12 — Buoyancy Behaviour

1. Receive solid density and fluid density.
2. If $\text{solid_density} > \text{fluid_density}$, return SINKS.
3. Else if $\text{solid_density} < \text{fluid_density}$, return FLOATS.
4. Otherwise return SUSPENDED.

Algorithm 13 — Square Root Using Newton-Raphson Method

1. Receive number.
2. If $\text{number} < 0$, return -1 .
3. If $\text{number} == 0$, return 0 .
4. Set $\text{old_value} = \text{number}$.
5. Using a do-while loop:
 - Calculate
 $\text{new_value} = 0.5 \times (\text{old_value} + \text{number} / \text{old_value})$
 - Calculate
 $\text{difference} = \text{new_value} - \text{old_value}$
 - If difference is negative, multiply it by -1 .
 - Set $\text{old_value} = \text{new_value}$.
6. Repeat while $\text{difference} \geq \text{TOLERANCE}$.
7. Return new_value .

3. Test Case Table

Results below use $\text{PI} = 3.1416$ and $\text{GRAVITY} = 9.8$.

T C	Operation	Input	Expected Output	Actual Output	Result
1	Sphere Area	$r=1$	12.566400 m ²	12.566400 m ²	Pass
2	Sphere Volume	$r=1$	4.188800 m ³	4.188800 m ³	Pass
3	Cylinder Area	$r=1, h=2$	18.849600 m ²	18.849600 m ²	Pass

4	Cylinder Volume	r=1, h=2	6.283200 m ³	6.283200 m ³	Pass
5	Cone Area	r=3, h=4	75.398400 m ²	75.398400 m ²	Pass
6	Cone Volume	r=3, h=4	37.699200 m ³	37.699200 m ³	Pass
7	Cuboid Area	l=2,b=3,h=4	52.000000 m ²	52.000000 m ²	Pass
8	Cuboid Volume	l=2,b=3,h=4	24.000000 m ³	24.000000 m ³	Pass
9	Circular Base	r=2	12.566400 m ²	12.566400 m ²	Pass
10	Cuboid Base	l=2,b=3	6.000000 m ²	6.000000 m ²	Pass
11	Base Area/Sphere	Sphere	Not defined	Appropriate message	Pass
12	Mass	q=1000, cuboid 2×3×4	24000 kg	24000 kg	Pass
13	Weight	q=1000, sphere r=1	41050.240000 N	41050.240000 N	Pass
14	Potential Energy Boundary	elevation=0	0 J	0 J	Pass
15	Pressure	q=1000, cylinder r=1,h=2	19600 Pa	19600 Pa	Pass
16	Buoyant Force	fluid q=1000, sphere	41050.240000 N	41050.240000 N	Pass
17	Buoyancy	solid=800, fluid=1000	Floats	Floats	Pass
18	Buoyancy Boundary	solid=1000, fluid=1000	Suspended	Suspended	Pass
19	Buoyancy	solid=1200, fluid=1000	Sinks	Sinks	Pass
20	Invalid Radius	r=0 then r=1	Error; request again	Error; request again	Pass
21	Invalid Height	h=-2 then h=2	Error; request again	Error; request again	Pass
22	Invalid Density	q=0 then 1000	Error; request again	Error; request again	Pass
23	Invalid Elevation	-10 then 0	Error; request again	Error; request again	Pass
24	Invalid Solid	5 then 1	Error; request again	Error; request again	Pass
25	Invalid Menu	0 then 10	Error; menu repeats	Error; menu repeats	Pass

Boundary interpretation: Since dimensions and densities must be **greater than zero**, 0 is the invalid boundary. Elevation may be zero, so 0 m is its valid boundary.

4. solid.h

```
#ifndef SOLID_H
#define SOLID_H
```

```

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

/* Geometry functions */
double sphere_area(double radius);
double sphere_volume(double radius);

double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double
height);
double cuboid_volume(double length, double breadth, double
height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* Physics functions */
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double
fluid_density);

/* Mathematical utility */
double calculate_square_root(double number);

#endif

```

5.math_utils.c

```
#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
        return -1;

    if (number == 0)
        return 0;

    old_value = number;

    do
    {
        new_value = 0.5 * (old_value + number / old_value);

        difference = new_value - old_value;

        if (difference < 0)
            difference = -difference;

        old_value = new_value;

    } while (difference >= TOLERANCE);

    return new_value;
}
```

6.geometry.c

```
#include "solid.h"

double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}
```

```

}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)
{
    double slant_height;

    slant_height =
        calculate_square_root(radius * radius + height *
height);

    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double
height)
{
    return 2 * (length * breadth +
                breadth * height +
                length * height);
}

double cuboid_volume(double length,
                    double breadth,
                    double height)
{

```

```

        return length * breadth * height;
    }

double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}

```

7. physics.c

```

#include <stdio.h>
#include "solid.h"

int get_physics_solid()
{
    int solid;

    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice. Enter 1 to 4.\n");

    } while (solid < 1 || solid > 4);

    return solid;
}

double get_positive_value()
{

```

```

double value;

do
{
    scanf("%lf", &value);

    if (value <= 0)
        printf("Invalid input. Enter a value greater than
zero: ");

    } while (value <= 0);

    return value;
}

double get_solid_volume(int solid)
{
    double radius, height, length, breadth;

    switch (solid)
    {
        case SPHERE:
            printf("Enter radius (m): ");
            radius = get_positive_value();

            return sphere_volume(radius);

        case CYLINDER:
            printf("Enter radius (m): ");
            radius = get_positive_value();

            printf("Enter height (m): ");
            height = get_positive_value();

            return cylinder_volume(radius, height);

        case CONE:
            printf("Enter radius (m): ");
            radius = get_positive_value();

            printf("Enter height (m): ");
            height = get_positive_value();

            return cone_volume(radius, height);
    }
}

```

```

        case CUBOID:
            printf("Enter length (m): ");
            length = get_positive_value();

            printf("Enter breadth (m): ");
            breadth = get_positive_value();

            printf("Enter height (m): ");
            height = get_positive_value();

            return cuboid_volume(length, breadth, height);
    }

    return 0;
}

double calculate_mass(double density)
{
    int solid;
    double volume;

    solid = get_physics_solid();

    volume = get_solid_volume(solid);

    return density * volume;
}

double calculate_weight()
{
    double density, mass;

    printf("Enter solid density (kg/m^3): ");
    density = get_positive_value();

    mass = calculate_mass(density);

    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    printf("Enter solid density (kg/m^3): ");

```

```

    density = get_positive_value();

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid;
    double radius, height;
    double length, breadth;
    double volume, contact_area;

    do
    {
        solid = get_physics_solid();

        if (solid == SPHERE)
        {
            printf("Pressure on base cannot be calculated "
                   "for a sphere.\n");
        }

    } while (solid == SPHERE);

    if (solid == CYLINDER)
    {
        printf("Enter radius (m): ");
        radius = get_positive_value();

        printf("Enter height (m): ");
        height = get_positive_value();

        volume = cylinder_volume(radius, height);
        contact_area = circular_base_area(radius);
    }
    else if (solid == CONE)
    {
        printf("Enter radius (m): ");
        radius = get_positive_value();

        printf("Enter height (m): ");
        height = get_positive_value();
    }
}

```

```

        volume = cone_volume(radius, height);
        contact_area = circular_base_area(radius);
    }
    else
    {
        printf("Enter length (m): ");
        length = get_positive_value();

        printf("Enter breadth (m): ");
        breadth = get_positive_value();

        printf("Enter height (m): ");
        height = get_positive_value();

        volume = cuboid_volume(length, breadth, height);
        contact_area = cuboid_base_area(length, breadth);
    }

    return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int solid;
    double volume;

    solid = get_physics_solid();

    volume = get_solid_volume(solid);

    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density,
                      double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;

    else if (solid_density < fluid_density)
        return FLOATS;

    else
        return SUSPENDED;
}

```

8.main.c

```
#include <stdio.h>
#include "solid.h"

int main()
{
    int choice, solid, behaviour;

    double radius, height;
    double length, breadth;
    double density, fluid_density;
    double elevation, result;

    do
    {
        printf("\n=====\\n");
        printf("        SOLIDS GEOMETRY AND PHYSICS\\n");
        printf("=====\\n");

        printf("1. Surface Area\\n");
        printf("2. Volume\\n");
        printf("3. Base Area\\n");
        printf("4. Mass\\n");
        printf("5. Weight\\n");
        printf("6. Gravitational Potential Energy\\n");
        printf("7. Pressure on the Base\\n");
        printf("8. Buoyant Force\\n");
        printf("9. Buoyancy Behaviour\\n");
        printf("10. Exit\\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice < 1 || choice > 10)
        {
            printf("Invalid menu choice. Enter 1 to 10.\\n");
            continue;
        }

        switch (choice)
```

```

{
    case 1:
        do
        {
            printf("Select solid "
                "(1-Sphere, 2-Cylinder, "
                "3-Cone, 4-Cuboid): ");

            scanf("%d", &solid);

            if (solid < 1 || solid > 4)
                printf("Invalid solid choice.\n");

        } while (solid < 1 || solid > 4);

        if (solid == SPHERE)
        {
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);

                if (radius <= 0)
                    printf("Radius must be greater
than zero.\n");

            } while (radius <= 0);

            result = sphere_area(radius);
        }

        else if (solid == CYLINDER)
        {
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
            } while (radius <= 0);

            do
            {
                printf("Enter height (m): ");
                scanf("%lf", &height);
            } while (height <= 0);
        }
    }
}

```

```

        result = cylinder_area(radius, height);
    }

else if (solid == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    result = cone_area(radius, height);
}

else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    result =
        cuboid_area(length, breadth, height);
}

printf("Total Surface Area = %.6f m^2\n",

```

```

        result);
    break;

case 2:

    do
    {
        printf("Select solid "
               "(1-Sphere, 2-Cylinder, "
               "3-Cone, 4-Cuboid): ");

        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");

    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        result = sphere_volume(radius);
    }

    else if (solid == CYLINDER)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);

        result =
            cylinder_volume(radius, height);
    }

```

```

    }

    else if (solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);

        result = cone_volume(radius, height);
    }

    else
    {
        do
        {
            printf("Enter length (m): ");
            scanf("%lf", &length);
        } while (length <= 0);

        do
        {
            printf("Enter breadth (m): ");
            scanf("%lf", &breadth);
        } while (breadth <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);

        result =
            cuboid_volume(length, breadth,
height);
    }

    printf("Volume = %.6f m^3\n", result);

```

```

        break;

case 3:

    do
    {
        printf("Select solid "
               "(1-Sphere, 2-Cylinder, "
               "3-Cone, 4-Cuboid): ");

        scanf("%d", &solid);

        if (solid < 1 || solid > 4)
            printf("Invalid solid choice.\n");

    } while (solid < 1 || solid > 4);

    if (solid == SPHERE)
    {
        printf("Base area is not defined "
               "for a sphere.\n");
        break;
    }

    if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);

            if (radius <= 0)
                printf("Invalid radius.\n");

        } while (radius <= 0);

        result = circular_base_area(radius);
    }
    else
    {
        do
        {
            printf("Enter length (m): ");
            scanf("%lf", &length);
        } while (length <= 0);
    }

```

```

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
} while (breadth <= 0);

result =
    cuboid_base_area(length, breadth);
}

printf("Base Area = %.6f m^2\n", result);
break;

case 4:

do
{
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater "
            "than zero.\n");

} while (density <= 0);

result = calculate_mass(density);

printf("Mass = %.6f kg\n", result);
break;

case 5:

result = calculate_weight();

printf("Weight = %.6f N\n", result);
break;

case 6:

do
{
    printf("Enter elevation (m): ");
    scanf("%lf", &elevation);

```

```
        if (elevation < 0)
            printf("Elevation cannot be negative.
\n");
```

```
    } while (elevation < 0);

    result = potential_energy(elevation);

    printf("Potential Energy = %.6f J\n",
        result);
    break;
```

case 7:

```
do
{
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater "
            "than zero.\n");

} while (density <= 0);

result = calculate_pressure(density);

printf("Pressure = %.6f Pa\n", result);
break;
```

case 8:

```
do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);

    if (fluid_density <= 0)
        printf("Density must be greater "
            "than zero.\n");

} while (fluid_density <= 0);

result = buoyant_force(fluid_density);
```

```

        printf("Buoyant Force = %.6f N\n", result);
        break;

    case 9:

        do
        {
            printf("Enter solid density (kg/m^3): ");
            scanf("%lf", &density);

            if (density <= 0)
                printf("Invalid density.\n");

        } while (density <= 0);

        do
        {
            printf("Enter fluid density (kg/m^3): ");
            scanf("%lf", &fluid_density);

            if (fluid_density <= 0)
                printf("Invalid density.\n");

        } while (fluid_density <= 0);

        behaviour =
            determine_buoyancy(density,
                               fluid_density);

        if (behaviour == SINKS)
            printf("The solid SINKS.\n");

        else if (behaviour == FLOATS)
            printf("The solid FLOATS.\n");

        else
            printf("The solid remains SUSPENDED.\n");

        break;

    case 10:
        printf("Program terminated.\n");
        break;

}

```

```
    } while (choice != 10);

    return 0;
}
```

9. Compilation

```
Last login: Sun Sep 13 15:07:58 on ttys000
[aaryanrajput@aaryans-macbook ~ % nano solid.h
[aaryanrajput@aaryans-macbook ~ % nano math_utils.c
[aaryanrajput@aaryans-macbook ~ % nano geometry.c
[aaryanrajput@aaryans-macbook ~ % nano physics.c
[aaryanrajput@aaryans-macbook ~ % nano main.c
[aaryanrajput@aaryans-macbook ~ % gcc main.c geometry.c physics.c math_utils.c -o solids
[aaryanrajput@aaryans-macbook ~ % █
```

10. Execution

Test: Cone Surface Area

```
[aaryanrajput@aaryans-macbook ~ % ./solids

=====
      SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1
Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3
Enter radius (m): 3
Enter height (m): 4
Total Surface Area = 75.398400 m^2
```

Test: Cuboid Mass

```
=====
SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 4
Enter solid density (kg/m^3): 1000

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 4
Enter length (m): 2
Enter breadth (m): 3
Enter height (m): 4
Mass = 24000.000000 kg
```

Test: Invalid Input

```
=====
SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1
Select solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 1
Enter radius (m): 0
Radius must be greater than zero.
Enter radius (m): -2
Radius must be greater than zero.
Enter radius (m): 1
Total Surface Area = 12.566400 m^2
```

Test: Buoyancy Behaviour

```
=====
SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 800
Enter fluid density (kg/m^3): 1000
The solid FLOATS.
```

For equal densities:

```
=====
SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 1000
Enter fluid density (kg/m^3): 1000
The solid remains SUSPENDED.
```

For greater solid density:

```
=====
SOLIDS GEOMETRY AND PHYSICS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 1200
Enter fluid density (kg/m^3): 1000
The solid SINKS.
```

11. Result

The modular C project implements all **nine calculation categories** and the Exit option using **switch-case**, iterative input validation, call-by-value functions, and separate source modules. It calculates geometry and physics quantities in SI units, uses the Newton-Raphson method instead of `math.h`, handles invalid and boundary inputs, and follows the stated restrictions against arrays, pointers, recursion, structures, unions, enumerations and `typedef`.